

Problem A. Prime-Weighted Segments

The problem asks us to maximize the minimum score among at most k segments.

Since the feasibility of a score X is monotonic, if a score is achievable then any smaller score is also achievable, so we can use Binary Search on the Answer.

We first precompute the number of distinct prime factors, denoted by $\omega(x)$, for every integer up to the maximum value in the array using a modified Sieve of Eratosthenes.

This allows us to evaluate the score of any gcd value in $O(1)$ time during the checks.

To determine whether a target score X is achievable, we process the array from left to right and maintain the gcd of the current segment.

As a segment grows, its gcd can only decrease or stay the same. Therefore, the number of distinct prime factors of the gcd can only decrease or stay the same as well, which means the segment score is monotonic and never increases when we extend the segment.

Using this observation, we greedily extend the current segment while its score remains at least X . If adding the next element keeps the score $\geq X$, we continue.

Otherwise, we close the current segment and start a new one from that element.

If a single element already has a score smaller than X , then X is impossible to achieve. After processing the whole array, if the number of constructed segments is less than or equal to k , then X is feasible.

The greedy strategy is optimal because extending a segment can never increase its score.

So whenever extending the current segment would make the score drop below X , that segment must end there in any valid partition.

Therefore, the greedy check produces the minimum possible number of segments for the chosen threshold X .

The complexity is $O(M \log \log M + N \log M \log(\max_score))$, where M is the maximum value in the array.

The sieve preprocessing takes $O(M \log \log M)$, and each feasibility check takes $O(N \log M)$ due to gcd computations.

The memory usage is $O(M)$ for storing the precomputed prime factor counts.

Problem B. Portals of the Multiverse

The problem asks us to process Q queries on an $N \times M$ grid. For each query, we need to check two conditions for a given rectangular sector (r_1, c_1) to (r_2, c_2) :

Portal Dominance: The number of portals (#) must be strictly greater than the number of floor tiles (.).

Dimensional Balance: The number of portals must have exactly 3 divisors.

Key Insights:

1. Identifying Numbers with Exactly 3 Divisors: A number has exactly 3 divisors if and only if it is the square of a prime number. Example: 4 (divisors: 1, 2, 4), 9 (divisors: 1, 3, 9), 25 (divisors: 1, 5, 25). To handle this efficiently, we can use a Sieve of Eratosthenes to find primes up to $\sqrt{N \times M}$ (which is $\sqrt{10^6} = 1000$) and then mark their squares as valid.

2. Counting Portals in a Rectangle:

Since we have up to $5 \cdot 10^5$ queries, counting characters one by one for each query would be $O(Q \cdot N \cdot M)$, which is too slow. We use 2D Prefix Sums to answer each query in $O(1)$.

Let $\text{pref}[i][j]$ be the number of # in the rectangle from $(1, 1)$ to (i, j) .

The number of # in a rectangle (r_1, c_1, r_2, c_2) is calculated as:

$$\text{count} = \text{pref}[r_2][c_2] - \text{pref}[r_1 - 1][c_2] - \text{pref}[r_2][c_1 - 1] + \text{pref}[r_1 - 1][c_1 - 1]$$

The number of floor tiles (.) is simply the total area of the rectangle minus the count of #:

$$\text{area} = (r_2 - r_1 + 1) \times (c_2 - c_1 + 1)$$

$$\text{floor_count} = \text{area} - \text{count}$$

Solution Steps:

Precompute: Use a sieve to find primes up to 1000 and store the squares of these primes in a boolean array or set.

Build Prefix Sum: Construct the 2D prefix sum array for the grid where # contributes 1 and . contributes 0.

Process Queries: For each query:

Calculate `portal_count` using the 2D prefix sum formula. Calculate `floor_count`. Check if `portal_count > floor_count` and `is_three_divisors[portal_count]` is true. Output YES or NO.

Complexity:

Time Complexity: Precomputing primes: $O(L \log L)$ where $L = 1000$. Building Prefix Sum: $O(N \times M)$.

Answering Queries: $O(Q)$. Total: $O(N \times M + Q)$.

Space Complexity: $O(N \times M)$ to store the prefix sum array.

Problem D. Abkarino's Approval

Prerequisites: Digit DP, Number Theory (Prime Factorization), Prefix Sum Array logic.

1. The Core Observation

We are asked to count how many numbers in the range $[L, R]$ have a valid sum of digits. The range can be up to 10^{100} , meaning the numbers can have up to 100 digits. This immediately screams **Digit DP**.

Like all range queries, we use the prefix difference property:

$$\text{Answer} = f(R) - f(L) + \text{isValid}(L)$$

where $f(X)$ counts the number of valid integers in the range $[0, X]$.

Because the maximum number of digits is 100, the maximum possible sum of digits is $100 \times 9 = 900$. This small maximum sum is the key to heavily optimizing the problem.

2. Precomputing Prime Factorizations

A sum S is considered "valid" if for every prime factor of S , its exponent e satisfies two conditions:

1. $e \leq 9$.
2. The e -th bit of the given mask M is set (`mask & (1 << e) != 0`).

Since the sum S will never exceed 900, doing prime factorization on the fly during the DP is a waste of time. We can precompute the exponents of the prime factorization for all numbers from 1 to 900 globally before processing any test cases. This makes our `valid(S, M)` check virtually $O(1)$.

3. The Unconstrained DP Trick (State Independence)

In a standard Digit DP, our state is usually `dp(idx, current_sum, tight)`. However, clearing a $100 \times 900 \times 2$ memoization table for every single query is too slow if the number of test cases T is large.

Notice what happens when `tight == 0` (meaning the prefix we have built is strictly smaller than the prefix of X). From this point onward, we can pick **any** digit from 0 to 9 for the remaining positions. The number of ways to finish the string only depends on two things:

1. `len`: The number of digits left to place.
2. `rem_sum`: The exact sum those remaining digits need to produce.

This is completely independent of the query limit X and the mask M !

We can globally precompute a 2D array `dp[len][sum]`, which represents the number of ways to choose `len` digits such that their sum is exactly `sum`. The transition is a simple bounded knapsack:

$$\text{dp}[\text{len}][\text{sum}] = \sum_{d=0}^9 \text{dp}[\text{len} - 1][\text{sum} - d]$$

4. The Optimized Query Execution

With `dp[len][sum]` precomputed, answering a query for an upper bound X becomes incredibly fast.

We recursively traverse the digits of X maintaining the `current_sum` and the `tight` flag.

- If `tight == 1`: We iterate the next digit d from 0 up to the limit of the current character of X , and recursively call the function. There are at most 100×10 tight states per query.
- If `tight == 0`: We do **not** recurse! We simply iterate over all possible sums i that the remaining suffix can produce ($0 \leq i \leq \text{len} \times 9$). If `valid(current_sum + i, M)` is true, we directly add `dp[len][i]` to our answer.

By immediately terminating the recursion on non-tight branches, we completely avoid the need to memoize the query-specific states!

5. Complexity

- **Global Precomputations:**
 - Precomputing `dp[100][900]`: $\mathcal{O}(N \times \text{MAX_SUM} \times 10) \approx 9 \times 10^5$ operations.
 - Factorizing $1 \dots 900$: $\mathcal{O}(\text{MAX_SUM} \times \sqrt{\text{MAX_SUM}}) \approx 2.7 \times 10^4$ operations.
 - These run exactly once.
- **Per Query ($f(X)$):**
 - We only ever follow the single "tight" path down the string (depth 100).
 - At each step where a branch becomes non-tight, we loop over possible remaining sums (≤ 900).
 - Thus, evaluating $f(X)$ takes at most $\mathcal{O}(N \times \text{MAX_SUM}) \approx 90,000$ operations.
- **Overall Time Complexity:** $\mathcal{O}(T \times N \times \text{MAX_SUM})$, which executes in a few milliseconds and easily shatters the time limit.
- **Space Complexity:** $\mathcal{O}(N \times \text{MAX_SUM})$ for the global DP table, taking less than 1 MB of memory.

Problem E. The Architects Game

Key Observations

- Tensions between consecutive digits form piles in a combinatorial game. The game is Moore's Nim-2: each move reduces up to two piles, and each chosen pile is reduced by an independent positive amount.
- A position is losing iff every binary digit's sum modulo 3 across all tension values equals zero.
- Since tensions are in the range $[0, 9]$, only the lowest 4 binary bits matter. We track four modulo-3 counters.
- Use digit DP to count losing positions in the range $[L, R]$. Alice wins if the position is not losing.

Solution

Algorithm: Digit DP to count losing positions.

1. For each number in $[L, R]$, compute the tensions as the absolute differences between consecutive digits.
2. Analyze the binary representation of each tension. For bit position $j \in \{0, 1, 2, 3\}$, sum the j -th bits of all tensions.
3. A position is losing iff all four bit-sums are divisible by 3.
4. Use digit DP to count losing positions in $[0, R]$ and $[0, L - 1]$. The answer for a range is the total number in that range minus the count of losing positions.

DP State:

- *pos*: current digit index processed from left to right.
- *tight*: boolean flag indicating whether the prefix still equals the upper bound.
- *started*: boolean flag indicating whether we have placed a non-zero digit yet (handles leading zeros).
- *prev*: the value of the previous digit (undefined when *started* = 0).
- b_0, b_1, b_2, b_3 : the accumulated sum modulo 3 for each of the four binary columns.

Transition: Place digit d at position *pos*. If *started* = 1, compute the tension $t = |d - \text{prev}|$ and update each counter $b_j \pmod 3$ where $j \in \{0, 1, 2, 3\}$.

Base case: At the end of the number, return 1 if $b_0 = b_1 = b_2 = b_3 = 0$ (losing position), else return 0.

Complexity

Time: $\mathcal{O}(D \times 2 \times 2 \times 11 \times 81)$ per bound query, where $D \leq 19$ is the maximum number of digits. Across $t \leq 100$ test cases, this executes well within the time limit.

Space: $\mathcal{O}(D \times 2 \times 2 \times 11 \times 81) \approx 68'000$ integers for the memoization table, which easily fits in memory.

Notes

- **Single-digit numbers:** Have no pairs of consecutive digits, hence no tensions. By definition, they are losing positions since all bit-sums b_j remain 0.
- **Multi-test handling:** Read t using a **while** (t-) loop structure. Clear or reset the memoization table states between different bound queries.
- **Avoiding overflow:** Use unsigned 64-bit integers (**unsigned long long** in C++) for range values. When computing $L - 1$, handle the edge case $L = 1$ separately to avoid underflow.
- **Edge cases:** Ensure validation covers ranges where $L = R$, maximum boundary values (10^{18}), and numbers with alternating or repeated digits.

Problem F. W-Pattern

A naive brute-force approach iterating over all possible 5 indices takes $\mathcal{O}(N^5)$ time, which immediately results in Time Limit Exceeded (TLE). A prefix/suffix array approach reduces the time to $\mathcal{O}(N)$ but requires $\mathcal{O}(N)$ auxiliary space and involves complex implementation.

This simplifies to the mathematical equation:

$$\text{Profit} = P[t_1] - 2 \cdot P[t_2] + 2 \cdot P[t_3] - 2 \cdot P[t_4] + P[t_5]$$

The optimal approach uses a State Machine Dynamic Programming concept. We can think of the W-pattern formation as a sequence of 5 required states. We maintain 5 variables, each representing the maximum value achieved for that specific state up to the current index i :

dp_1 : maximum value of $P[t_1]$

dp_2 : maximum value of $P[t_1] - 2 \cdot P[t_2]$

dp_3 : maximum value of $P[t_1] - 2 \cdot P[t_2] + 2 \cdot P[t_3]$

dp_4 : maximum value of $P[t_1] - 2 \cdot P[t_2] + 2 \cdot P[t_3] - 2 \cdot P[t_4]$

dp_5 : maximum value of $P[t_1] - 2 \cdot P[t_2] + 2 \cdot P[t_3] - 2 \cdot P[t_4] + P[t_5]$

As we iterate through the array, we update each state using the optimal value of the previous state from the previous step, enforcing the strictly increasing time condition ($t_1 < t_2 < t_3 < t_4 < t_5$).

By initializing the states with a very small number (e.g., -10^{18}), we ensure invalid incomplete states are ignored. The time complexity is $O(N)$, and the space complexity is strictly $O(1)$.

Problem G. Cursed Library

Prerequisites: DSU on Tree (SACK), Segment Tree, Offline Queries.

Analysis: A naive solution would involve running a DFS to collect all nodes in the subtree for each query, inserting their chapter numbers into a set or array, sorting them, and finding the longest consecutive sequence in the range $[L, R]$. This takes $O(Q \cdot N \log N)$ and will immediately yield a Time Limit Exceeded (TLE) verdict.

Some participants might attempt to flatten the tree using a DFS Euler Tour (mapping subtrees to contiguous subarray intervals) and use **Mo's Algorithm**. However, answering a query inside Mo's Algorithm would require either a fast $O(1)$ update and $O(\sqrt{N})$ query, or vice-versa. Given the strict Time Limit (2.0s) and Memory Limit (128 MB), an $O(N\sqrt{N})$ approach will struggle heavily and likely TLE or MLE.

The Intended Solution: We can process the queries *offline* and traverse the tree using **DSU on Tree (SACK)**. To quickly answer the range queries about contiguous chapter sequences, we maintain a global **Segment Tree** over the domain of chapter numbers $[1, \max C]$.

1. The Segment Tree State: Each node in our Segment Tree represents a range of chapter values $[l, r]$. It must store four variables to support merging:

- **len:** The length of the interval ($r - l + 1$).
- **pref:** The length of the longest contiguous sequence of present chapters starting exactly at l .
- **suff:** The length of the longest contiguous sequence of present chapters ending exactly at r .
- **max_c:** The maximum length of any contiguous sequence of present chapters entirely within $[l, r]$.

When merging a **left** child and a **right** child, the new states become:

- $\text{res.pref} = \text{left.pref} + (\text{left.pref} == \text{left.len} ? \text{right.pref} : 0)$
- $\text{res.suff} = \text{right.suff} + (\text{right.suff} == \text{right.len} ? \text{left.suff} : 0)$
- $\text{res.max_c} = \max(\text{left.max_c}, \text{right.max_c}, \text{left.suff} + \text{right.pref})$

2. DSU on Tree (SACK) Integration: We process the tree recursively. For a node u :

1. Recursively solve for all *light* children of u , rolling back (clearing) their contributions from the global Segment Tree after each.
2. Recursively solve for the *heavy* child of u , **keeping** its contribution in the Segment Tree.
3. Traverse the light children of u again, as well as u itself, and add their chapters to the Segment Tree.
4. Now, the Segment Tree perfectly reflects the set of chapters present in the subtree of u . We iterate through all offline queries associated with u . For a query $[L, R]$, we perform a range query on the Segment Tree. The answer is the `max_c` of the returned node.
5. If u is a light child of its parent, roll back (remove) all elements in u 's subtree from the Segment Tree to prevent state bleed.

To ensure $\mathcal{O}(\log(\max C))$ updates, we maintain a global frequency array. We only update the Segment Tree at index C_i to 1 (active) when its frequency changes from $0 \rightarrow 1$, and update it to 0 (inactive) when its frequency drops from $1 \rightarrow 0$.

Complexity:

- **Time:** In SACK, each node is added and removed at most $\mathcal{O}(\log N)$ times. Each addition/removal updates the Segment Tree in $\mathcal{O}(\log(\max C))$. Answering queries takes $\mathcal{O}(\log(\max C))$. Overall Time Complexity: $\mathcal{O}(N \log N \log(\max C) + Q \log(\max C))$.
- **Memory:** The memory heavily relies on the flat arrays and the Segment Tree, bounded by $\mathcal{O}(N + \max C + Q)$, safely passing the strict 128 MB limit.

Problem H. Filtered Substring Intersections

Build a Suffix Automaton (SAM) for S to compactly represent all distinct substrings. Each SAM state represents a set of substrings that appear at the same set of end positions. Propagate the occurrence count up the suffix link tree to know how many times each substring appears in S .

Build the KMP prefix function for P and convert it into a DFA that tracks the longest matched prefix of P .

Define $DP(u, j)$ as the number of valid distinct substrings reachable from SAM state u with KMP match length j . For each character c , transition to $v = \text{next}(u, c)$. If $\text{cnt}(v) \geq K$ and the new KMP state $j' < |P|$ (no full match of P yet), then add $DP(v, j')$. Process SAM states in reverse topological order (longest to shortest).

Time complexity: $\mathcal{O}(|S| \cdot |P|)$ states, but practically much faster due to limited SAM branching.

Problem I. Goe plus plus

Key Observation. All coordinates are integers. If we can separate the points of Color A from the points of Color B and Color C using a convex polygon whose vertices have half-integer coordinates, then every point strictly inside or strictly outside is determined exactly. A clean way to achieve strict containment with integer-coordinate polygon vertices is to expand each point (x, y) into the four axis-aligned neighbors $(x \pm 1, y)$ and $(x, y \pm 1)$, and take the convex hull of the expanded set.

Main Idea. Iterate over all three pairs of colors $\{A, B\} \subseteq \{1, 2, 3\}$, with the remaining color being C . For each pair:

1. For each point (x, y) of Color A , add the four points $(x + 1, y)$, $(x - 1, y)$, $(x, y + 1)$, $(x, y - 1)$ to a set P_A . Compute the convex hull H_A of P_A . Do the same for Color B to obtain H_B .
2. Check whether the boundaries of H_A and H_B intersect. If they do, this pair fails.

3. Check whether every point of Color C lies strictly outside both H_A and H_B . If any point of Color C lies on or inside either hull, this pair fails.
4. If both checks pass, output H_A and H_B .

Correctness. Since all original points have integer coordinates and the hull vertices are also integers, a point (x, y) of Color A is strictly inside H_A if and only if all four neighbors were added to P_A : the convex hull of the expanded set guarantees a margin of at least 1 around each original point in the L^1 sense, which implies strict interior containment. A point of Color C at integer coordinates either falls strictly outside the hull (correct) or on/inside it (detected by the check). The boundary intersection test uses a standard Shamos–Hoey sweep-line algorithm on the edges of H_A and H_B .

Complexity. For each of the 3 color pairs, expanding points costs $O(n)$, computing convex hulls costs $O(n \log n)$, the sweep-line intersection test costs $O(m \log m)$ where m is the total number of hull edges, and the point-in-convex-polygon checks cost $O(n \log n)$ using binary search. The total complexity is $O(n \log n)$.

Problem J. Thanos Sort

Problem Analysis

Given an array A of even size N , we need to find the probability that a subsequence of length $K = N/2$, chosen uniformly at random, is non-decreasing. The result should be output modulo 998244353.

Combinatorics

The total number of ways to choose a subsequence of length $N/2$ is given by the binomial coefficient:

$$Q = \binom{N}{N/2} = \frac{N!}{(N/2)! \times (N/2)!}$$

The probability is calculated as $P \times Q^{-1} \pmod{998244353}$, where P represents the count of non-decreasing subsequences of length $N/2$.

Dynamic Programming

We define $dp[len][i]$ as the number of non-decreasing subsequences of length len that end at index i .

The transition is defined as:

$$dp[len][i] = \sum_{j < i, A[j] \leq A[i]} dp[len-1][j]$$

Base Case:

$$dp[1][i] = 1, \text{ for all } 0 \leq i < N$$

Final Answer:

$$P = \sum_{i=0}^{N-1} dp[N/2][i]$$

Optimization

The naive DP approach has a complexity of $O(N^3)$, which is inefficient for $N = 5000$. Since the transition involves a prefix sum over values where $A[j] \leq A[i]$, we can optimize this using a Fenwick Tree (Binary Indexed Tree).

- Coordinate compress the values of A into the range $[1, N]$.
- For each length l from 2 to $N/2$:
 - Use a Fenwick Tree to store the dp values of length $l-1$.
 - For each index i , $dp[l][i] = BIT.query(compressed_A[i])$.

Complexity

- **Time Complexity:** $O(N^2 \log N)$
- **Space Complexity:** $O(N)$

Problem K. String Parity Shift

We represent the characters 'A', 'B', and 'C' as numerical values 0, 1, and 2 respectively. The operation allows us to select two adjacent indices, i and $i+1$, and increment both of their values by 1 under modulo 3.

To find if string S can be transformed into string T , we need to find an invariant property that remains unchanged after any number of operations. Consider the alternating sum of the string array elements, where values at even indices are added and values at odd indices are subtracted:

$$\text{Sum} = \left(\sum_{j=0}^{n-1} (-1)^j \cdot \text{val}[j] \right) \bmod 3$$

When we apply the operation at index i , we add 1 to both S_i and S_{i+1} . Because one of these indices is even and the other is odd, one will add +1 to the alternating sum and the other will add -1. As a result, the total net change to the alternating sum modulo 3 is always 0.

Thus, the alternating sum modulo 3 is a strict invariant. If the alternating sum of string S modulo 3 equals the alternating sum of string T modulo 3, it is always possible to match the strings — and the answer is "YES". Otherwise, it is "NO".

Problem L. Sonic and Graphs

To find the maximum standard vehicle weight X , we can binary search the answer in the range $[0, \max(W_i)]$.

For a candidate weight X , we need to verify if all convoys can successfully reach their destinations on their assigned days, using only highways with $W_i \geq X$. We can check this efficiently offline.

Since highways are available on days $t < D_i$, a convoy traveling on day T_j can use highway i if $D_i > T_j$. To optimize the process:

1. We sort all highways descending by their collapse day D_i .
2. We sort all convoy queries descending by their travel day T_j .
3. We iterate through the sorted queries. For each query, we add to a Disjoint Set Union (DSU) all highways whose collapse day is strictly greater than the query's travel day, provided their weight limit $W_i \geq X$.
4. Then, we simply check if the source and destination of the current query are connected in the DSU.

Since we pre-sort the edges and queries once globally, checking a candidate weight X takes $O(M\alpha(N) + K)$ time. Total Time Complexity: $O(M \log M + K \log K + \log(\max W) \times (M\alpha(N) + K))$.

Problem M. Final Exams

The study time must be partitioned into exactly K positive study sessions.

The number of ways to divide N study hours into K positive parts equals:

$$\binom{N-1}{K-1}$$

Now consider the placement of the M playing hours.

Since there must be at least one playing hour between every two consecutive study sessions, we first reserve $K - 1$ playing hours for the internal gaps.

After reserving them, the remaining:

$$M - (K - 1)$$

playing hours can be distributed among the $K + 1$ positions:

- before the first study session,
- between consecutive study sessions,
- after the last study session.

Using stars and bars, the number of distributions equals:

$$\binom{M + 1}{K}$$

Therefore, the final answer is:

$$\binom{N - 1}{K - 1} \times \binom{M + 1}{K}$$

Special cases:

- If $K = 0$, the answer equals 1 only when $N = 0$.
- If $K > N$, the answer equals 0.
- If $M < K - 1$, the answer equals 0.

Because $K \leq 10^6$ while N and M may be as large as 10^{18} , compute combinations using multiplicative formulas with precomputed modular inverses.

The complexity is $O(K)$.

Problem N. Find the Path

Hint 1

For small constraints, this problem is solved efficiently using dynamic programming. If we let $dp(i, d)$ be the number of paths of length i ending with stone type d , how do we transition?

Hint 2

We can place a block of stone type d of length k , where $1 \leq k \leq d$. If we append this block, the remaining sequence to its left must have a length of $i - k$ and crucially, **must not** end with stone type d . Therefore, our recurrence relation is:

$$dp(i, d) = \sum_{k=1}^d \sum_{x \neq d} dp(i - k, x)$$

Hint 3

Notice that to compute the values for step i , we only need to look back at the previous 4 steps $(i - 1, i - 2, i - 3, i - 4)$ since the maximum value of d is 4. Because the state transitions linearly based on a fixed history, we can map this exact DP recurrence into Matrix Exponentiation to handle $n \leq 10^{18}$.

Solution

Since we need a sliding window of the last 4 states of our DP, and each state tracks 4 stone types, our state vector V_i will have a size of $4 \times 4 = 16$.

$$V_i = [dp(i, 1..4), \dots, dp(i - 3, 1..4)]^T$$

We construct a 16×16 transition matrix M such that $V_i = M \times V_{i-1}$. The top 4 rows of M represent calculating $dp(i, d)$. Following our recurrence:

$$\begin{aligned} dp(i, 1) &= \sum_{x \neq 1} dp(i - 1, x) \\ dp(i, 2) &= \sum_{x \neq 2} dp(i - 1, x) + \sum_{x \neq 2} dp(i - 2, x) \\ &\dots \end{aligned}$$

Thus, for a given d and step back k , we set $M[d - 1][(k - 1) \times 4 + (x - 1)] = 1$ for all $x \neq d$. The remaining 12 rows of M simply shift the history downward (e.g., $dp(i - 1)$ becomes the new $i - 2$ offset).

For each test case, we compute the base values V_4 manually using standard DP. If $n \leq 4$, we print the answer directly. If $n > 4$, we multiply V_4 by M^{n-4} using binary matrix exponentiation.

To process $t = 1000$ efficiently, precompute the binary powers of M globally exactly once. For each query, multiply the vector V_4 directly by the precomputed matrices.

Complexity

Time Complexity: $O(S^3 \log(\max N))$ once for matrix precomputations, where $S = 16$. For each test case, the vector-matrix multiplication takes $O(t \cdot S^2 \log N)$. Since $16^2 = 256$, operations are heavily minimized and will effortlessly run well within the 1-second limit.

Space / Memory Complexity: $O(S^2 \log(\max N))$ to store the precomputed matrices.